ICA Algorithms: Mathematical Theory Reference

Derivations, Pseudocode, and Analysis

Ratanakmuny NOUV
M2 Data Science — Université d'Évry

April 2026

Contents

1 ICA Problem Formulation **3**
1.1 The Generative Model . . . 3
1.2 Assumptions . . . 3
1.3 Inherent Ambiguities . . . 3
1.4 Identifiability Theorem . . . 3

2 Preprocessing: Whitening **3**
2.1 Why Whiten? . . . 3
2.2 Derivation . . . 4
2.3 Effect on the Mixing Matrix . . . 4

3 Likelihood and Score Function **4**
3.1 The Log-Likelihood of ICA . . . 4
3.2 The Score Function . . . 5
3.3 Choice of Score Function: Derivation . . . 5
3.4 Extended Infomax for Mixed Sources . . . 5

4 Gradient Derivation (Full Step-by-Step) **5**
4.1 First Term: $\log|\det V|$ . . . 5
4.2 Second Term: $\sum_j \log p_j(v_j^\top \mathbf{x})$ . . . 6
4.3 Full Gradient . . . 6
4.4 Simplification on Whitened Data . . . 6
4.5 Gradient Ascent Update . . . 6

5 Natural Gradient (Amari, 1998) **6**
5.1 Motivation . . . 6
5.2 Derivation . . . 7
5.3 Natural Gradient Update . . . 7

6 Symmetric Decorrelation **7**
6.1 Problem . . . 7
6.2 The Projection . . . 7
6.3 Computation via Eigendecomposition . . . 8

7 Algorithm 1: Infomax Batch (Bell & Sejnowski, 1995) **8**
7.1 Theory . . . 8
7.2 Pseudocode . . . 8

8 Algorithm 2: FastICA (Hyvärinen, 1999) **8**
8.1 Theory . . . 8
8.2 Negentropy . . . 8
8.3 Negentropy Approximation . . . 9
8.4 Optimization Problem . . . 9
8.5 Lagrangian Derivation . . . 9
8.6 Newton Iteration . . . 9
8.7 Final FastICA Update . . . 9
8.8 Pseudocode (Parallel FastICA) . . . 10

9 Algorithm 3: SGD-ICA **10**
9.1 Theory . . . 10
9.2 Mini-Batch Gradient . . . 10
9.3 Robbins-Monro Conditions . . . 10
9.4 Pseudocode . . . 11

10 Algorithm 4: EM-ICA (Our Contribution) **11**
10.1 Motivation . . . 11
10.2 Density Model . . . 11
10.3 E-Step: Responsibilities . . . 11
10.4 M-Step: MoG Parameter Updates . . . 12
10.5 Learned Score Function . . . 12
10.6 Pseudocode . . . 13
10.7 Why EM-ICA Works . . . 13

11 Amari Index **13**
11.1 Motivation . . . 13
11.2 The Performance Matrix . . . 13
11.3 The Amari Index Formula . . . 14
11.4 Understanding the Formula . . . 14
11.5 Properties . . . 14
11.6 Pseudocode . . . 15

12 Summary of All Algorithms **15**

1 ICA Problem Formulation

1.1 The Generative Model

We observe N samples $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ where each $\mathbf{x}_t \in \mathbb{R}^D$ is generated by the linear mixture:

$$\mathbf{x}_t = W\mathbf{z}_t, \tag{1}$$

with:

- $\mathbf{z}_t \in \mathbb{R}^D$: hidden source signals (what we want to recover),
- $W \in \mathbb{R}^{D\times D}$: unknown invertible mixing matrix,
- $V = W^{-1}$: the *unmixing matrix* we want to estimate.

Given V, we recover the sources as $\mathbf{y}_t = V\mathbf{x}_t$.

1.2 Assumptions

ICA requires three key assumptions:

1. **Statistical independence:** the sources are mutually independent:

$$p(\mathbf{z}) = \prod_{j=1}^{D} p_j(z_j).$$

2. **Non-Gaussianity:** *at most one* source can be Gaussian. This is the fundamental identifiability condition (proved below).
3. **Invertibility:** W is square and invertible.

1.3 Inherent Ambiguities

ICA has two unresolvable ambiguities:

Permutation. If P is any permutation matrix, then $W' = WP^{-1}$ and $\mathbf{z}' = P\mathbf{z}$ produce the same observations $\mathbf{x} = W'\mathbf{z}' = W\mathbf{z}$. **Meaning:** the ordering of recovered sources is arbitrary.

Sign/scaling. If Λ is a diagonal matrix, then $W' = W\Lambda^{-1}$ and $\mathbf{z}' = \Lambda\mathbf{z}$ produce identical observations. We remove the scaling ambiguity by fixing $\mathbb{E}[z_j^2] = 1$, but the *sign* remains (since ± 1 preserves unit variance).

Consequence. When evaluating ICA, we must match estimated sources to true sources via permutation and sign correction. The Amari index handles both automatically.

1.4 Identifiability Theorem

Theorem 1 (Darmois, 1953 / Comon, 1994)**.** *Let $\mathbf{x} = W\mathbf{z}$ where $\mathbf{z}$ has independent components. If at most one component of $\mathbf{z}$ is Gaussian, then W is identifiable up to permutation and scaling.*

Intuition. If two sources are Gaussian, any orthogonal rotation of them remains jointly Gaussian and independent. The likelihood is invariant under such rotations, so the gradient provides no signal for separation. This explains the **Gaussian failure case** in our experiments: all algorithms produce Amari > 0.3 when all sources are Gaussian.

2 Preprocessing: Whitening

2.1 Why Whiten?

On raw data, V has D^2 free parameters. After whitening, V becomes orthogonal, reducing the search space to $D(D-1)/2$ parameters — less than half.

2.2 Derivation

Let $\tilde{\mathbf{x}} = M(\mathbf{x} - \mathbb{E}[\mathbf{x}])$ be the whitened data. We want $\mathbb{E}[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^\top] = I$.

Step 1: Center the data. Define $\mathbf{x}_c = \mathbf{x} - \mathbb{E}[\mathbf{x}]$.

Step 2: Compute the covariance matrix:

$$\Sigma = \mathbb{E}[\mathbf{x}_c\mathbf{x}_c^\top] \approx \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_{c,i}\mathbf{x}_{c,i}^\top. \tag{2}$$

Step 3: Eigendecompose Σ:

$$\Sigma = E\Lambda E^\top, \tag{3}$$

where E is the matrix of eigenvectors and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_D)$.

Step 4: The whitening matrix is:

$$M = \Lambda^{-1/2}E^\top \quad \Longrightarrow \quad \tilde{\mathbf{x}} = \Lambda^{-1/2}E^\top\mathbf{x}_c. \tag{4}$$

Verification:

$$\begin{aligned}
\mathbb{E}[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^\top] &= \Lambda^{-1/2}E^\top\,\mathbb{E}[\mathbf{x}_c\mathbf{x}_c^\top]\,E\Lambda^{-1/2} && (5)\\
&= \Lambda^{-1/2}E^\top(E\Lambda E^\top)E\Lambda^{-1/2} && (6)\\
&= \Lambda^{-1/2}\Lambda\Lambda^{-1/2} = I. \quad \checkmark && (7)
\end{aligned}$$

2.3 Effect on the Mixing Matrix

After whitening, the effective mixing matrix $\tilde{W} = MW$ satisfies:

$$\tilde{W}\tilde{W}^\top = MWW^\top M^\top = I, \tag{8}$$

assuming the true sources have unit variance ($\mathbb{E}[\mathbf{z}\mathbf{z}^\top] = I$). Therefore $\tilde{W}$ is **orthogonal**, and so is $V = \tilde{W}^{-1} = \tilde{W}^\top$.

Key consequence. On whitened data, $V^{-\top} = V$. This simplifies the gradient formula and eliminates the need for matrix inversion.

3 Likelihood and Score Function

3.1 The Log-Likelihood of ICA

Assume the sources $\mathbf{z}$ have density $p(\mathbf{z}) = \prod_j p_j(z_j)$. Since $\mathbf{x} = W\mathbf{z}$, the change of variables formula gives:

$$p_{\mathbf{x}}(\mathbf{x}) = p_{\mathbf{z}}(V\mathbf{x}) \cdot |\det V|, \tag{9}$$

where $V = W^{-1}$.

Taking the log and using independence:

$$\log p_{\mathbf{x}}(\mathbf{x}) = \log|\det V| + \sum_{j=1}^{D} \log p_j(v_j^\top\mathbf{x}), \tag{10}$$

where $v_j^\top$ is the j-th row of V.

For N i.i.d. samples, the log-likelihood per sample is:

$$\boxed{\ell(V) = \log|\det V| + \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{D} \log p_j(v_j^\top\mathbf{x}_i)} \tag{11}$$

3.2 The Score Function

The score function of source j is defined as the derivative of its log-density:

$$g_j(y) := \frac{d}{dy} \log p_j(y) = \frac{p_j'(y)}{p_j(y)}. \tag{12}$$

Why the score matters. The score function encodes the *shape* of the density. It is zero at the modes of p_j and changes sign across them. The ICA gradient update uses g_j to push estimated sources toward the correct distribution.

3.3 Choice of Score Function: Derivation

A common assumption is that super-Gaussian sources follow the density:

$$p(y) = \frac{1}{\pi \cosh(y)}. \tag{13}$$

Step 1: Take the log:

$$\log p(y) = -\log \cosh(y) - \log \pi. \tag{14}$$

Step 2: Differentiate:

$$g(y) = \frac{d}{dy} \log p(y) = -\frac{\sinh(y)}{\cosh(y)} = \boxed{-\tanh(y)}. \tag{15}$$

Contrast vs. score. The course (Section 5.7.4) introduces the contrast function $G(u) = \log \cosh(u)$ with derivative $G'(u) = +\tanh(u)$. Note that:

$$G(u) = -\log p(u) + \text{const} \quad \Longrightarrow \quad G'(u) = -g(u). \tag{16}$$

So the course's $g = G' = +\tanh$ is the *contrast derivative*, while our $g = -\tanh$ is the *true score*. Both conventions give valid ICA solutions (sign ambiguity).

3.4 Extended Infomax for Mixed Sources

The score $g = -\tanh$ assumes super-Gaussian sources. For sub-Gaussian sources (kurtosis < 0), this fails. The Extended Infomax lee1999 uses kurtosis-based switching:

$$g_j(y) = \begin{cases} -\tanh(y) & \text{if } \kappa_j > 0 \quad \text{(super-Gaussian)} \\ -y + \tanh(y) & \text{if } \kappa_j \leq 0 \quad \text{(sub-Gaussian)} \end{cases} \tag{17}$$

where κ_j is the excess kurtosis (kurtosis minus 3):

$$\kappa_j = \frac{\mathbb{E}[(y_j - \mu_j)^4]}{\sigma_j^4} - 3. \tag{18}$$

4 Gradient Derivation (Full Step-by-Step)

Starting from (11), we compute $\nabla_V \ell(V)$.

4.1 First Term: $\log |\det V|$

Using the matrix identity:

$$\frac{\partial}{\partial V} \log |\det V| = (V^{-1})^\top = V^{-\top}. \tag{19}$$

Proof sketch. From Jacobi's formula: $\frac{\partial \det V}{\partial V_{ij}} = \mathrm{cof}(V_{ij})$, and $(V^{-1})_{ji} = \mathrm{cof}(V_{ij})/\det V$. Dividing by $\det V$:

$$\frac{\partial \log|\det V|}{\partial V_{ij}} = \frac{1}{\det V}\mathrm{cof}(V_{ij}) = (V^{-1})_{ji} = (V^{-\top})_{ij}. \tag{20}$$

4.2 Second Term: $\sum_j \log p_j(v_j^\top \mathbf{x})$

For a single sample $\mathbf{x}_i$, with $y_{j,i} = v_j^\top \mathbf{x}_i$, we use the chain rule:

$$\frac{\partial}{\partial V_{jk}} \log p_j(y_{j,i}) = \underbrace{\frac{p_j'(y_{j,i})}{p_j(y_{j,i})}}_{g_j(y_{j,i})} \cdot \underbrace{\frac{\partial y_{j,i}}{\partial V_{jk}}}_{x_{i,k}}. \tag{21}$$

So the partial derivative with respect to V_{jk} is $g_j(y_{j,i}) \cdot x_{i,k}$. Writing this in matrix form, the contribution of sample i to row j is $g_j(y_{j,i})\mathbf{x}_i^\top$. Summing over j gives $g(\mathbf{y}_i)\mathbf{x}_i^\top$ where $g(\mathbf{y}_i) = [g_1(y_{1,i}), \ldots, g_D(y_{D,i})]^\top$.

Summing over all samples and normalizing:

$$\frac{\partial}{\partial V}\left[\frac{1}{N}\sum_{i,j} \log p_j(v_j^\top \mathbf{x}_i)\right] = \frac{1}{N}\sum_{i=1}^{N} g(\mathbf{y}_i)\mathbf{x}_i^\top = \frac{1}{N} g(Y)X^\top, \tag{22}$$

where $Y = VX \in \mathbb{R}^{D\times N}$.

4.3 Full Gradient

Combining both terms:

$$\nabla_V \ell(V) = V^{-\top} + \frac{1}{N} g(Y)X^\top. \tag{23}$$

4.4 Simplification on Whitened Data

On whitened data, V is orthogonal, so $V^{-\top} = V$:

$$\nabla_V \ell(V) = V + \frac{1}{N} g(Y)X^\top. \tag{24}$$

4.5 Gradient Ascent Update

Master update rule (used by Infomax, SGD-ICA, EM-ICA):

$$V \leftarrow V + \eta\left(V + \frac{1}{N} g(Y)X^\top\right) \tag{25}$$

followed by symmetric decorrelation $V \leftarrow (VV^\top)^{-1/2}V$.

5 Natural Gradient (Amari, 1998)

5.1 Motivation

The Euclidean gradient (23) treats all components of V equally, but the space of invertible matrices has a non-Euclidean geometry. The natural gradient accounts for this by rescaling the gradient using the Fisher information metric.

5.2 Derivation

The natural gradient is defined as:

$$\tilde{\nabla}_V \ell = (F^{-1}) \nabla_V \ell, \tag{26}$$

where F is the Fisher information matrix. For ICA, Amari amari1998 showed that multiplying the Euclidean gradient by $V^\top V$ gives the natural gradient:

$$\tilde{\nabla}_V \ell = \left(V^{-\top} + \frac{1}{N} g(Y) X^\top \right) V^\top V. \tag{27}$$

Expanding:

$$\begin{aligned}
\tilde{\nabla}_V \ell &= V^{-\top} V^\top V + \frac{1}{N} g(Y) X^\top V^\top V && (28)\\
&= V + \frac{1}{N} g(Y) (VX)^\top V && (29)\\
&= V + \frac{1}{N} g(Y) Y^\top V && (30)\\
&= \left(I + \frac{1}{N} g(Y) Y^\top \right) V. && (31)
\end{aligned}$$

5.3 Natural Gradient Update

$$\boxed{V \leftarrow V + \eta \left(I + \frac{1}{N} g(Y) Y^\top \right) V} \tag{32}$$

Advantages:

- No matrix inversion (no $V^{-\top}$).
- **Equivariance**: convergence speed is independent of W.
- Typically converges faster than Euclidean gradient.

Note. On whitened data with symmetric decorrelation, the Euclidean and natural gradients give similar results because V is constrained to be orthogonal. We used the Euclidean form in our implementation.

6 Symmetric Decorrelation

6.1 Problem

After a gradient step, V may drift from orthogonality. We need to project it back onto the orthogonal group $O(D) = \{V : VV^\top = I\}$.

6.2 The Projection

The symmetric decorrelation is:

$$V \leftarrow (VV^\top)^{-1/2} V. \tag{33}$$

Verification that the result is orthogonal. Let $V' = (VV^\top)^{-1/2} V$. Then:

$$\begin{aligned}
V'V'^\top &= (VV^\top)^{-1/2} VV^\top (VV^\top)^{-1/2} && (34)\\
&= (VV^\top)^{-1/2} (VV^\top)(VV^\top)^{-1/2} && (35)\\
&= (VV^\top)^{-1/2+1-1/2} = (VV^\top)^0 = I. \quad \checkmark && (36)
\end{aligned}$$

6.3 Computation via Eigendecomposition

To compute $(VV^\top)^{-1/2}$, we use:

1. Compute $A = VV^\top$.
2. Eigendecompose $A = UDU^\top$ where $D = \mathrm{diag}(d_1, \ldots, d_D)$.
3. Compute $A^{-1/2} = U\mathrm{diag}(1/\sqrt{d_1}, \ldots, 1/\sqrt{d_D})U^\top$.

Why "symmetric"? This projection treats all components of V equally, unlike Gram-Schmidt which extracts them sequentially.

7 Algorithm 1: Infomax Batch (Bell & Sejnowski, 1995)

7.1 Theory

Infomax maximizes the mutual information between inputs and outputs of a nonlinear neural network. For a specific choice of nonlinearity, this is equivalent to maximum likelihood estimation with a specific source prior. The update is exactly the batch gradient ascent (24).

7.2 Pseudocode

Algorithm 1 Infomax Batch ICA

Require: Whitened data $X \in \mathbb{R}^{D\times N}$, learning rate η, max iterations T, tolerance ϵ
Ensure: Unmixing matrix V

```
Initialize V ← I_D + 0.01 N(0, I)                ▷ near identity + noise
V ← (VV^⊤)^{-1/2} V                              ▷ enforce orthogonality
for t = 1 to T do
    Y ← VX                                       ▷ current source estimates
    Compute g(Y) via adaptive score (17)         ▷ per-source kurtosis
    ∇ ← V + (1/N) g(Y) X^⊤                       ▷ gradient
    V_new ← V + η∇                               ▷ gradient step
    V_new ← (V_new V_new^⊤)^{-1/2} V_new         ▷ symmetric decorrelation
    if max |V_new V^⊤ − I| < ε then
        break
    end if
    V ← V_new
end for
return V
```

Complexity. Each iteration is $O(DN + D^3)$: matrix multiply dominates for large N, eigendecomposition for large D.

8 Algorithm 2: FastICA (Hyvärinen, 1999)

8.1 Theory

FastICA maximizes an approximation of negentropy (non-Gaussianity measure), one component at a time.

8.2 Negentropy

Definition 1 (Negentropy)**.** *Let* $\mathbf{y}$ *be a random vector. Its negentropy is:*

$$J(\mathbf{y}) = H(\mathbf{y}_{gauss}) - H(\mathbf{y}), \tag{37}$$

where H *is differential entropy and* $\mathbf{y}_{gauss}$ *is a Gaussian with the same covariance.*

Property. $J(\mathbf{y}) \geq 0$, with equality iff $\mathbf{y}$ is Gaussian. Therefore, maximizing non-Gaussianity $\equiv$ maximizing negentropy.

8.3 Negentropy Approximation

Exact negentropy requires the full density, which is unknown. Hyvärinen proposed the approximation:

$$J(y) \approx c\,(\mathbb{E}[G(y)] - \mathbb{E}[G(\nu)])^2\,, \tag{38}$$

where c is a constant, G is a non-quadratic function (e.g., $G(u) = \log\cosh(u)$), and ν is a standard Gaussian.

8.4 Optimization Problem

FastICA maximizes J over unit-norm vectors:

$$\max_{\mathbf{w}} \Big(\mathbb{E}[G(\mathbf{w}^\top \mathbf{x})] - \mathbb{E}[G(\nu)]\Big)^2 \quad \text{subject to } \|\mathbf{w}\| = 1. \tag{39}$$

8.5 Lagrangian Derivation

The Lagrangian is:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbb{E}[G(\mathbf{w}^\top \mathbf{x})] - \lambda(\|\mathbf{w}\|^2 - 1). \tag{40}$$

Setting $\nabla_{\mathbf{w}} \mathcal{L} = 0$:

$$\mathbb{E}[\mathbf{x}\, g(\mathbf{w}^\top \mathbf{x})] - 2\lambda \mathbf{w} = 0, \tag{41}$$

where $g = G'$. So the stationarity condition is:

$$\mathbb{E}[\mathbf{x}\, g(\mathbf{w}^\top \mathbf{x})] = 2\lambda \mathbf{w}. \tag{42}$$

This says the vector $\mathbb{E}[\mathbf{x}\, g(\mathbf{w}^\top \mathbf{x})]$ must be parallel to $\mathbf{w}$.

8.6 Newton Iteration

Define $F(\mathbf{w}) = \mathbb{E}[\mathbf{x}\, g(\mathbf{w}^\top \mathbf{x})] - 2\lambda \mathbf{w}$. Newton's method for solving $F = 0$:

$$\mathbf{w}_{\text{new}} = \mathbf{w} - [J_F(\mathbf{w})]^{-1} F(\mathbf{w}), \tag{43}$$

where J_F is the Jacobian:

$$J_F(\mathbf{w}) = \mathbb{E}[\mathbf{x}\mathbf{x}^\top g'(\mathbf{w}^\top \mathbf{x})] - 2\lambda I. \tag{44}$$

Approximation. On whitened data, $\mathbb{E}[\mathbf{x}\mathbf{x}^\top] = I$, so we approximate:

$$\mathbb{E}[\mathbf{x}\mathbf{x}^\top g'(\mathbf{w}^\top \mathbf{x})] \approx \mathbb{E}[g'(\mathbf{w}^\top \mathbf{x})] \cdot I. \tag{45}$$

8.7 Final FastICA Update

Substituting into the Newton step and simplifying:

$$\boxed{\mathbf{w} \leftarrow \mathbb{E}[\mathbf{x}\, g(\mathbf{w}^\top \mathbf{x})] - \mathbb{E}[g'(\mathbf{w}^\top \mathbf{x})]\, \mathbf{w}} \tag{46}$$

followed by normalization $\mathbf{w} \leftarrow \mathbf{w}/\|\mathbf{w}\|$.

8.8 Pseudocode (Parallel FastICA)

Algorithm 2 FastICA (Parallel)

Require: Whitened data X, max iterations T, tolerance ϵ
Ensure: Unmixing matrix V

```
Initialize V with random orthogonal matrix
for t = 1 to T do
    Y ← VX
    for each row v_j of V do
        v_j ← E[x g(v_j^⊤ x)] − E[g'(v_j^⊤ x)] v_j
    end for
    V ← (VV^⊤)^{−1/2} V                                   ▷ symmetric decorrelation
    if max |diag(V^(t) V^(t−1)^⊤) − 1| < ε then
        break
    end if
end for
return V
```

Why so fast? Newton's method has *cubic convergence*: the error is cubed at each iteration. This is why FastICA typically converges in 10–20 iterations, vs. hundreds for gradient methods. In our experiments, FastICA was ∼500× faster.

9 Algorithm 3: SGD-ICA

9.1 Theory

Replace the full expectation in (24) with a mini-batch estimate. This enables online learning and scales to large datasets.

9.2 Mini-Batch Gradient

For a mini-batch X_B of size B, the stochastic gradient is:

$$\hat{\nabla}_V \ell_B = V + \frac{1}{B} g(Y_B) X_B^\top, \quad Y_B = V X_B. \tag{47}$$

This is an unbiased estimator:

$$\mathbb{E}[\hat{\nabla}_V \ell_B] = \nabla_V \ell. \tag{48}$$

9.3 Robbins-Monro Conditions

For stochastic approximation to converge, the learning rate η_t must satisfy:

$$\sum_{t=1}^{\infty} \eta_t = \infty, \qquad \sum_{t=1}^{\infty} \eta_t^2 < \infty. \tag{49}$$

Intuition.

- First condition: sufficient exploration (steps don't shrink too fast).
- Second condition: noise cancellation (steps shrink fast enough for variance to vanish).

The schedule $\eta_t = \eta_0/(1 + t/\tau)$ satisfies both:

- $\sum \eta_t \sim \int dt/t = \infty$.
- $\sum \eta_t^2 \sim \int dt/t^2 < \infty$.

9.4 Pseudocode

Algorithm 3 SGD-ICA

Require: Whitened data X, initial lr η_0, decay τ, batch size B, epochs E
Ensure: Unmixing matrix V

```
Initialize V ← I_D + 0.01 N(0, I)
V ← (V V^⊤)^{-1/2} V
t ← 0
for epoch = 1 to E do
    Shuffle sample indices
    for each mini-batch X_B do
        Y_B ← V X_B
        Compute g(Y_B) via adaptive score
        ∇ ← V + (1/B) g(Y_B) X_B^⊤
        η_t ← η_0/(1 + t/τ)                    ▷ decaying lr
        V ← V + η_t ∇
        V ← (V V^⊤)^{-1/2} V
        t ← t + 1
    end for
end for
return V
```

Why SGD often beats batch. The stochastic noise acts as implicit regularization, helping escape poor local optima. In our experiments, SGD-ICA had lower variance across runs (0.010 ± 0.003) than Infomax batch (0.036 ± 0.076).

10 Algorithm 4: EM-ICA (Our Contribution)

10.1 Motivation

Extended Infomax has only 2 possible score shapes ($-\tanh$ or $-y + \tanh$). When sources share similar kurtosis but differ in density shape (e.g., real photographs), this fails. EM-ICA *learns* the score function from data via a Mixture of Gaussians.

10.2 Density Model

Each source density is modeled as a Mixture of K Gaussians:

$$p_j(z_j) = \sum_{k=1}^{K} \pi_{jk} \mathcal{N}(z_j \mid \mu_{jk}, \sigma_{jk}^2), \tag{50}$$

where π_{jk} are mixture weights ($\sum_k \pi_{jk} = 1$), μ_{jk} are component means, and σ_{jk}^2 are component variances.

10.3 E-Step: Responsibilities

For each source j, component k, and sample i, the posterior probability that sample i came from component k is:

$$r_{jk,i} = \frac{\pi_{jk} \mathcal{N}(y_{j,i} \mid \mu_{jk}, \sigma_{jk}^2)}{\sum_{k'=1}^{K} \pi_{jk'} \mathcal{N}(y_{j,i} \mid \mu_{jk'}, \sigma_{jk'}^2)}. \tag{51}$$

Implementation note. We compute this in log-space using the log-sum-exp trick to prevent numerical underflow.

10.4 M-Step: MoG Parameter Updates

Standard EM updates for a Gaussian mixture:

$$N_{jk} = \sum_i r_{jk,i} \tag{52}$$

$$\pi_{jk}^{\text{new}} = N_{jk}/B \tag{53}$$

$$\mu_{jk}^{\text{new}} = \frac{1}{N_{jk}} \sum_i r_{jk,i} y_{j,i} \tag{54}$$

$$(\sigma_{jk}^2)^{\text{new}} = \frac{1}{N_{jk}} \sum_i r_{jk,i} (y_{j,i} - \mu_{jk}^{\text{new}})^2 \tag{55}$$

Stochastic stability. Since we process mini-batches, these estimates are noisy. We use exponential moving average (EMA):

$$\theta \leftarrow (1-\alpha)\theta_{\text{old}} + \alpha\theta_{\text{batch}}, \quad \alpha = 0.1. \tag{56}$$

10.5 Learned Score Function

The score of a MoG density is the true $(\log p)'$:

$$g_j(y) = \frac{d}{dy} \log p_j(y) = \frac{p_j'(y)}{p_j(y)}. \tag{57}$$

Derivation. The derivative of a single Gaussian:

$$\frac{d}{dy}\mathcal{N}(y \mid \mu, \sigma^2) = -\frac{y-\mu}{\sigma^2}\mathcal{N}(y \mid \mu, \sigma^2). \tag{58}$$

Therefore:

$$p_j'(y) = -\sum_k \pi_{jk} \frac{y-\mu_{jk}}{\sigma_{jk}^2} \mathcal{N}(y \mid \mu_{jk}, \sigma_{jk}^2). \tag{59}$$

Dividing by $p_j(y)$ and recognizing the responsibility $r_{jk}(y) = \pi_{jk}\mathcal{N}(\cdot)/p_j(y)$:

$$\boxed{g_j(y) = -\sum_{k=1}^{K} r_{jk}(y) \, \frac{y-\mu_{jk}}{\sigma_{jk}^2}} \tag{60}$$

This learned score replaces the fixed $-\tanh$ in the gradient formula.

10.6 Pseudocode

Algorithm 4 EM-ICA

Require: Whitened data X, MoG components K, lr η_0, decay τ, batch size B, epochs E
Ensure: Unmixing matrix V

```
Initialize V near identity + symmetric decorrelation
Initialize MoG: π_jk = 1/K, μ_jk ~ N(0, 0.5^2), σ^2_jk = 1
t ← 0
for epoch = 1 to E do
    for each mini-batch X_B do
        Y_B ← V X_B
        E-step: Compute responsibilities r_{jk,i} (log-sum-exp)
        M-step (MoG): Update (π, μ, σ^2) with EMA
        Learned score: g_j(y_i) ← −Σ_k r_{jk,i}(y_i − μ_jk)/σ^2_jk
        ∇ ← V + (1/B) g(Y_B) X_B^⊤
        Gradient clipping if ‖∇‖ > 10
        η_t ← η_0/(1 + t/τ)
        V ← V + η_t ∇
        V ← (V V^⊤)^{−1/2} V
        t ← t + 1
    end for
end for
return V, MoG parameters
```

10.7 Why EM-ICA Works

The key insight is that kurtosis-switching gives *identical* scores for sources with similar kurtosis. The MoG learns a *different shape* for each source, capturing density features beyond kurtosis.

Empirical evidence. On real photographs (Astronaut, Camera, Coins) where all sources have $\kappa \approx -1.2$, EM-ICA achieves Amari $= 0.034$ vs. ≈ 0.086 for kurtosis-switching methods — a $2.5\times$ improvement.

11 Amari Index

11.1 Motivation

We need a metric that:

1. Handles the permutation ambiguity of ICA.
2. Handles the scaling/sign ambiguity.
3. Returns 0 for perfect separation, > 0 otherwise.
4. Is invariant to scaling of rows/columns.

11.2 The Performance Matrix

Given the estimated unmixing V and true mixing W, define:

$$C = VW. \tag{61}$$

Ideal case. If $V = W^{-1}$ perfectly, then $C = I$. More generally, perfect separation allows permutation and scaling, so C should be a *permutation-scaling matrix*: exactly one non-zero entry per row and column.

11.3 The Amari Index Formula

The Amari index amari1996 is:

$$r(C) = \frac{1}{2D(D-1)} \left[\sum_{i=1}^{D} \left(\sum_{j=1}^{D} \frac{|c_{ij}|}{\max_k |c_{ik}|} - 1 \right) + \sum_{j=1}^{D} \left(\sum_{i=1}^{D} \frac{|c_{ij}|}{\max_k |c_{kj}|} - 1 \right) \right] \tag{62}$$

11.4 Understanding the Formula

Term 1: row-wise. For each row i, normalize by the maximum of that row, then sum:

$$\sum_{j=1}^{D} \frac{|c_{ij}|}{\max_k |c_{ik}|}. \tag{63}$$

If row i has exactly one non-zero entry (perfect case), this sum equals 1 (the max itself, divided by itself). Subtracting 1 gives 0.

If row i has multiple non-zero entries, each contributes a fraction between 0 and 1, so the sum is > 1, and after subtracting 1, we get a positive contribution to the Amari index.

Term 2: column-wise. Same logic, but normalizing by column maxima. This handles the case where C has "clean columns" but "dirty rows" or vice versa.

Why both terms? Consider a pathological C with one row being zero and another having 2 non-zero entries. The row term would catch this but the column term might not, or vice versa. Including both ensures robustness.

Normalization. The factor $1/(2D(D-1))$ bounds $r \in [0, 1]$:

- Maximum of each term: each row/column contribution is at most $D - 1$, and there are D rows/columns, so max per term is $D(D-1)$.
- Sum of both terms: $\leq 2D(D-1)$.
- Dividing gives $r \leq 1$.

11.5 Properties

Proposition 1 (Scale invariance)**.** *$r(\Lambda_1 C \Lambda_2) = r(C)$ for any diagonal matrices Λ_1, Λ_2 with non-zero entries.*

Proof sketch. Row normalization cancels the left scaling Λ_1. Column normalization cancels the right scaling Λ_2. □

Proposition 2 (Permutation invariance)**.** *$r(PC) = r(CP) = r(C)$ for any permutation matrix P.*

Why these matter. Scale invariance handles the ICA sign/scale ambiguity. Permutation invariance handles the ordering ambiguity. Together, $r(C) = 0$ iff C is a permutation-scaling matrix, i.e., iff separation is perfect.

11.6 Pseudocode

Algorithm 5 Amari Index Computation

Require: Estimated V, true W
Ensure: Amari index r

$C \leftarrow VW$
$C_{\text{abs}} \leftarrow |C|$
$\mathbf{r}_{\max} \leftarrow$ row-wise max of C_{abs}
$\mathbf{c}_{\max} \leftarrow$ column-wise max of C_{abs}
Term1 $\leftarrow \sum_i \left(\sum_j C_{\text{abs},ij} / r_{\max,i} - 1 \right)$
Term2 $\leftarrow \sum_j \left(\sum_i C_{\text{abs},ij} / c_{\max,j} - 1 \right)$
return $(\text{Term1} + \text{Term2})/(2D(D-1))$

12 Summary of All Algorithms

Algorithm	**Update**	**Score**	**Key property**
Infomax Batch	Full-batch gradient ascent	Kurtosis switching	Slow, exact gradient
FastICA	Fixed-point (approx. Newton)	$G = \log \cosh$	Cubic convergence, $\sim$500$\times$ faster
SGD-ICA	Mini-batch stochastic gradient	Kurtosis switching	Online, Robbins-Monro convergence
EM-ICA	Stochastic EM	**Learned MoG**	Adapts to density shapes, wins on real photos

References

[Amari et al.(1996)] S. Amari, A. Cichocki, and H.H. Yang. A new learning algorithm for blind signal separation. *NIPS*, 1996.

[Amari(1998)] S. Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.

[Bell and Sejnowski(1995)] A.J. Bell and T.J. Sejnowski. An information-maximization approach to blind separation. *Neural Computation*, 7(6):1129–1159, 1995.

[Hyvärinen(1999)] A. Hyvärinen. Fast and robust fixed-point algorithms for independent component analysis. *IEEE Trans. Neural Networks*, 10(3):626–634, 1999.

[Lee et al.(1999)] T.-W. Lee, M. Girolami, and T.J. Sejnowski. Independent component analysis using an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural Computation*, 11(2):417–441, 1999.